



# Ethernet and TCP/IP

Communication Protocols

# Ethernet

---

- ❖ A well established, flexible and scalable network technology
  - Used in networking and communication systems
  - Describes how network devices can format and transmit data
  - The working group [802](#) of IEEE has responsibility for Ethernet
    - E.g., IEEE 802.1, IEEE 802.2, IEEE 802.3, etc.
    - First standardization in 1983 as [IEEE 802.3](#)
  - Defines the Data Link Layer and Physical Layer of the OSI model
    - Supports a large number of physical media
  - Used in Local Area Networks (LAN) for transferring data with 10Mbps to 1Gbps
  - Used by different protocols in the network layer of the OSI model. E.g. IP in TCP/IP
  - The [OPEN Alliance](#) SIG (Special Interest Group) was founded in the automotive industry in 2011.
    - It works with the IEEE on the transformation of Automotive Ethernet into general standards



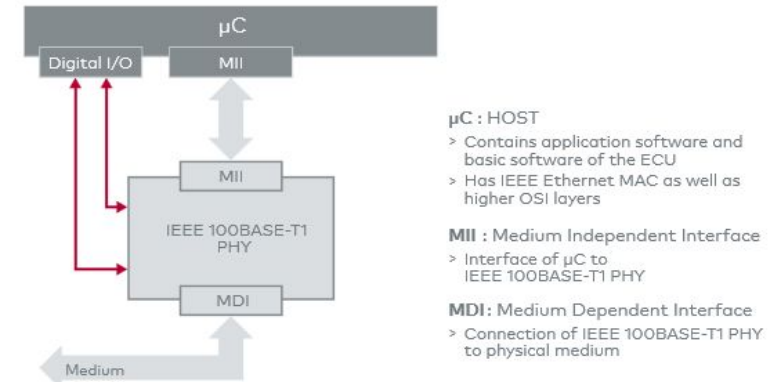
# Ethernet - Physical Layer

## ❖ In the physical layer, Ethernet defines:

- Cabling, connectors and electrical features of the layer
- Sending and receiving the bit streams over the medium
- Data encoding and decoding
- Collision detection

## ❖ Ethernet supports a large number of physical media

- Coaxial cable, twisted pair wires and optical fiber
- There are so many different standards ([Ethernet physical layer](#))
- Example standards: **10BASE-2**, **10BASE-T**, **100BASE-TX**, **100BASE-FX** and **1000BASE-T**
- Physical media are protocol-neutral; by providing a medium independent interface (MII)
  - Other transmission technologies can also be easily developed and adapted
- In the Automotive Ethernet **IEEE 100BASE-T1**, **IEEE 100BASE-TX** and **IEEE 1000BASE-T** are used



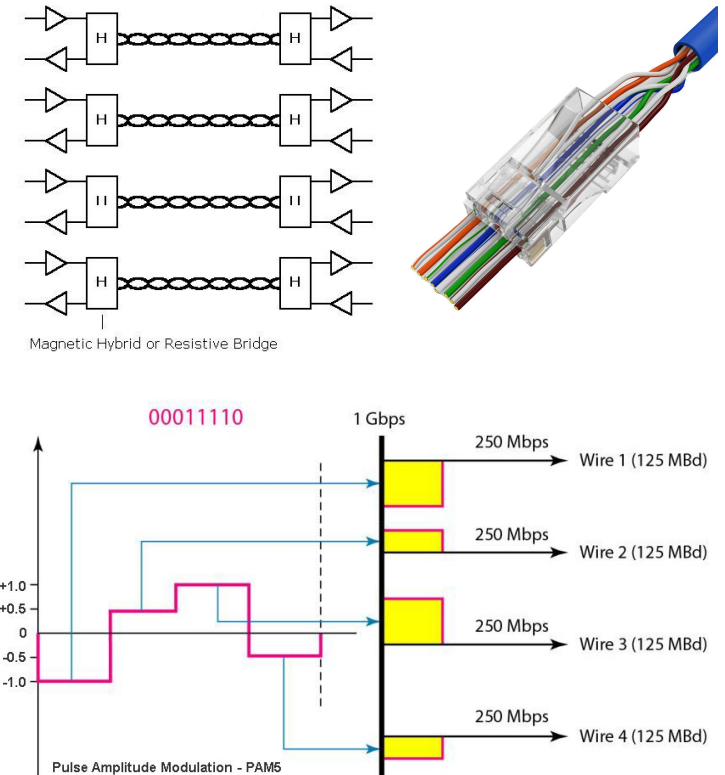
# Ethernet - Physical Layer (Automotive IEEE 100BASE-T1)

- ❖ Originally developed by Broadcom and standardized as IEEE 100BASE-T1
- ❖ The transformation into an IEEE standard was done by the Open Alliance SIG
- ❖ A twisted pair cable with symmetrical differential signaling is used
- ❖ IEEE 100BASE-T1 PHY uses 4B3B, 3B2T, and PAM3 methods to encode, decode and generate the differential voltages
- ❖ Data link layer sends 4 parallel bits with clock rate 25 MHz
- ❖ The PHY converts these 4 bits to 3 bits and increases the clock rate to 33.33 MHz to maintain the 100 Mbps bit rate
- ❖ Master-slave arrangement, slave is synchronized to the master clock
- ❖ Max distance: 15 meters; impedance: 90  $\Omega$  to 110  $\Omega$
- ❖ Supports full-duplex: to send; the value level is added and to receive, the value level is subtracted



# Ethernet - Physical Layer (IEEE 1000BASE-T)

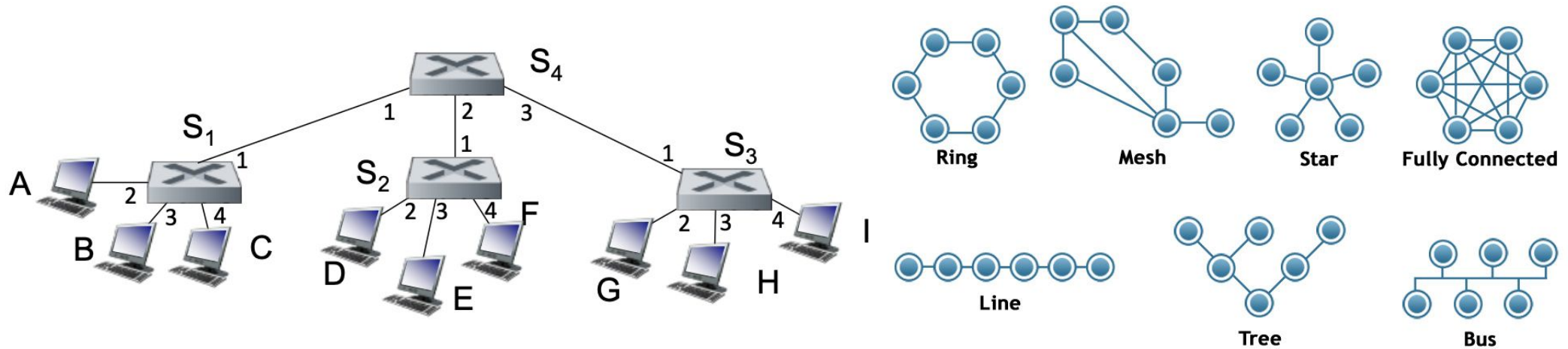
- ❖ Requires 4 channels of twisted wires
- ❖ Standardized Cat5e cables and RJ45 connectors are used
- ❖ Maximum length: 100 meters
- ❖ Symmetrical differential signaling is used
- ❖ On each channel symbol rate is: 125 Msps
- ❖ Each symbol carries two bits
- ❖ Bit rate of each channel is 250 Mbps
- ❖ Total bit rate of all 4 channels is 1000 Mbps
- ❖ Supports full-duplex: to send; the level of the value is added and to receive, the level of the value is subtracted
- ❖ Master-slave arrangement, slave is synchronized to the master clock.
  - Unlike 100BASE-T1, the roles are not fixed and can be negotiated by an auto negotiation mechanism





# Ethernet - Topology

- ❖ Ethernet supports point-to-point, bus, star and hybrid
- ❖ Bus is not fault tolerant, it is not used any more.
  - It was used by 10BASE-X standard
- ❖ Most basic topology is point to point.
  - Coupling elements like hub, switch or router are used to make other topologies



# Ethernet - Data Link Layer

## ❖ In the data link layer, Ethernet defines:

- Services and interfaces to the upper layer
- Addressing (MAC address)
- Media Access Control (CSMA/CD)
- Ethernet Framing
- Error Detection

## ❖ Node Addressing

- Every node has a unique MAC address (Medium Access Control Address)
- This address is a physical address which has been burnt in the NIC by the manufacturer
- Is a 6-byte address (48 bits - 12 hex). E.g. 18:db:f2:05:96:a5 - Dell Inc
  - The first 3 bytes are the unique identifier of the NIC manufacturer
  - The next 3 bytes are the unique identifier of the NIC

| OSI Model    |          | TCP/IP Stack        |  |
|--------------|----------|---------------------|--|
| Application  |          | Application         |  |
| Presentation |          |                     |  |
| Session      |          |                     |  |
| Transport    |          | Transport           |  |
| Network      |          | Internet            |  |
| Data Link    | Ethernet | Network Access/Link |  |
| Physical     |          |                     |  |

# Ethernet - Data Link Layer

---

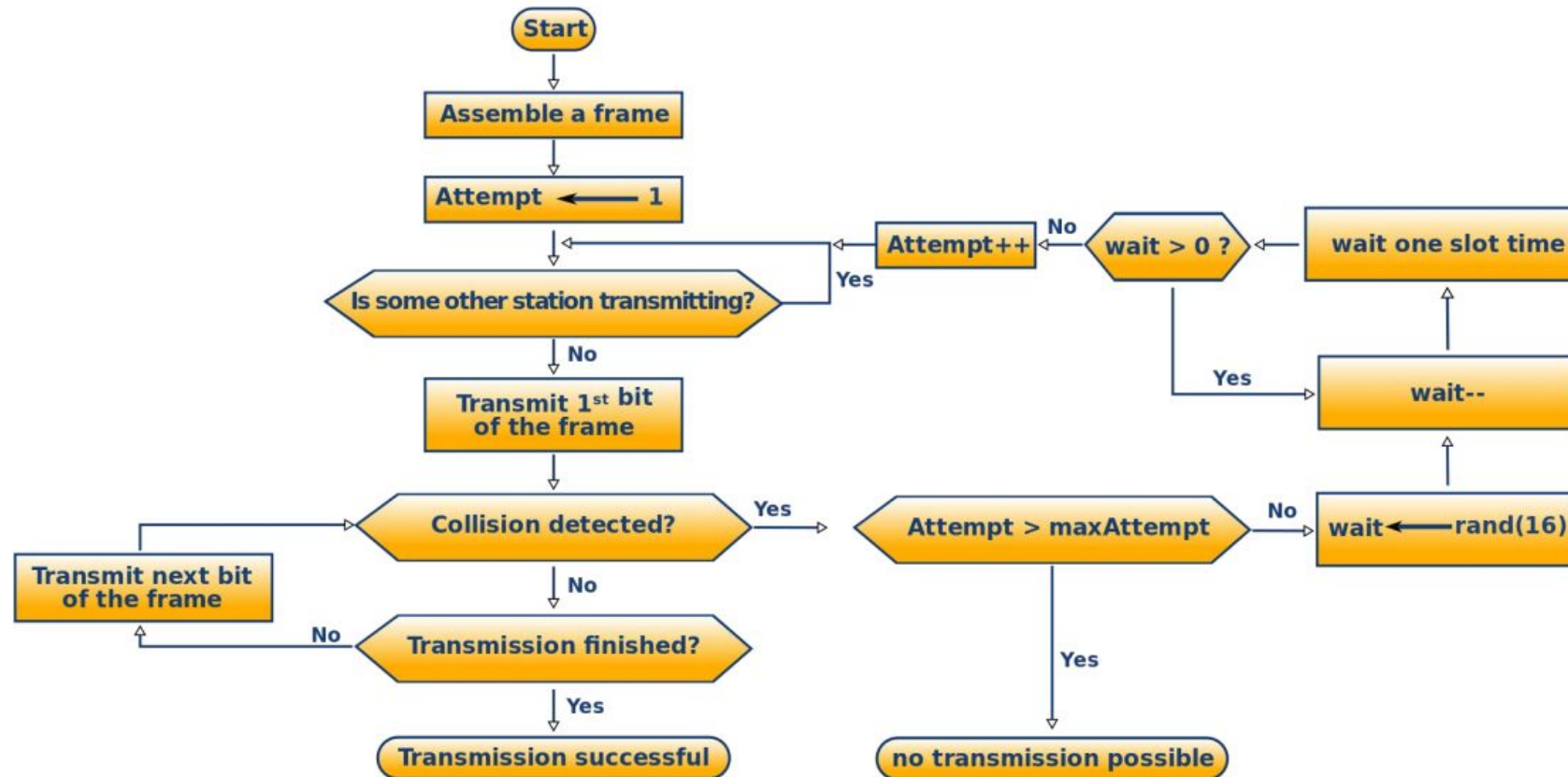
## ❖ Media Access Control (CSMA/CD)

- Carrier Sense Multiple Access/Collision Detection (CSMA/CD) method is used
- The Ethernet NIC, before sending, listens on the physical medium (carrier sense)
- If the medium is free, the Ethernet controller can begin its transmission
- Since multiple nodes may access to the bus at the same time (multiple access)
  - Collisions may occur on classic bus networks if two nodes begin sending
  - In the case of collision, each Ethernet NIC has a collision detection function which is used to stop the transmission.
  - To prevent a second collision, the node sends only after expiration of a random time.
  - Each sender must calculate this random time by itself
- Collisions normally don't occur on **IEEE 100BASE-T1, IEEE 100BASE-TX, and IEEE 1000BASE-T**. Because these is no bus a the physical media support full-duplex.



# Ethernet - Data Link Layer

## ❖ Media Access Control (CSMA/CD)



# Ethernet - Frame Format

---

- ❖ The IEEE defines different formats for Ethernet frames
- ❖ Ethernet II Frame is the most used one and it is directly used by Internet Protocol
- ❖ The automotive industry typically uses the Ethernet II frame

- ❖ Ethernet II Frame Fields: 

|                       |                 |                                   |                                 |                   |                              |                           |
|-----------------------|-----------------|-----------------------------------|---------------------------------|-------------------|------------------------------|---------------------------|
| Preamble<br>(7 bytes) | SFD<br>(1 byte) | Receiver MAC Address<br>(6 bytes) | Sender MAC Address<br>(6 bytes) | Type<br>(2 bytes) | Payload<br>(46 - 1500 bytes) | CRC Checksum<br>(4 bytes) |
|-----------------------|-----------------|-----------------------------------|---------------------------------|-------------------|------------------------------|---------------------------|

  - **Preamble**(7 bytes): A series of 56 bits alternating with 1 and 0 (0x55)
    - It is used for synchronisation. It basically is the sender's clock
  - **SFD**(1 byte): Start of frame delimiter (10101011)
    - Indicates to the receiver that the actual data is on its way
  - **Receiver MAC Address** (6 byte): The physical address of the receiver
  - **Sender MAC Address** (6 byte): The physical address of the sender

# Ethernet - Frame Format

## ❖ Ethernet II Frame Fields:

➤ **Type Field**(2 bytes): The upper layer protocol that uses the frame to send data.

- A code of 2 bytes is assigned to every protocol that is developed for Ethernet.

➤ **Payload**(46 - 1500 bytes): Contains the data to be sent

- The data received from the upper layer protocol
- If the data is less than 46 bytes, padding is used to meet the minimum length of the data

➤ **CRC Checksum**(4 bytes): A 4-byte CRC value

- It is calculated over the Sender MAC Address, Receiver MAC Address, Type, and Payload fields and therefore assures the integrity of MAC frame fields

Some examples of Type Field

| Type Field | Description                                       |
|------------|---------------------------------------------------|
| 0x0800     | Payload contains IPv4 packet                      |
| 0x0806     | Payload contains ARP packet                       |
| 0x22F0     | Audio/Video Transport Protocol (AVTP)             |
| 0x22EA     | Multiple Stream Registration Protocol (MSRP)      |
| 0x8100     | Ethernet VLAN Frame, Ether Type follows after tag |
| 0x86DD     | Payload contains IPv6 packet                      |
| 0x88F5     | Multiple VLAN Registration Protocol (MVRP)        |
| 0x88F6     | Multiple MAC Registration Protocol (MMRP)         |
| 0x88F7     | Precision Time Protocol (PTP)                     |

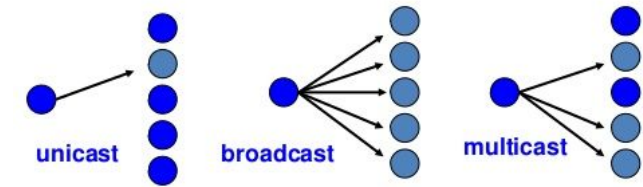


# Ethernet - Addressing

- ❖ In addition to unicasting, multicasting and broadcasting are supported

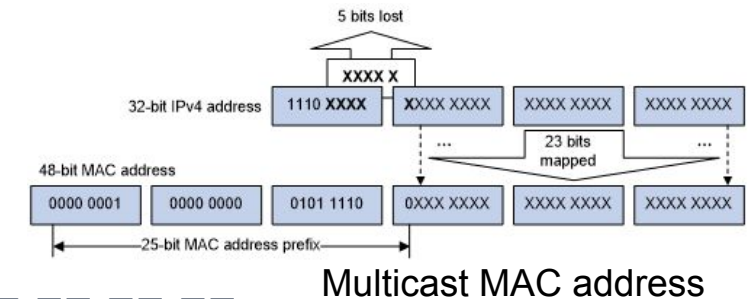
- ❖ Multicasting: Messages are delivered to multiple nodes

- 23 least significant bits of an IPv4 multicast address are mapped to the first 23 bits of destination MAC address of multicast frames.
- Multicast IP addresses are in the range of 224.0.0.0 – 239.255.255.255
- A node might receive unwanted multicast data at the data link layer
  - Such frames need to be filtered by the upper layer



- ❖ Broadcasting: Messages are delivered to all nodes

- The destination MAC address of the frames is FF:FF:FF:FF:FF:FF

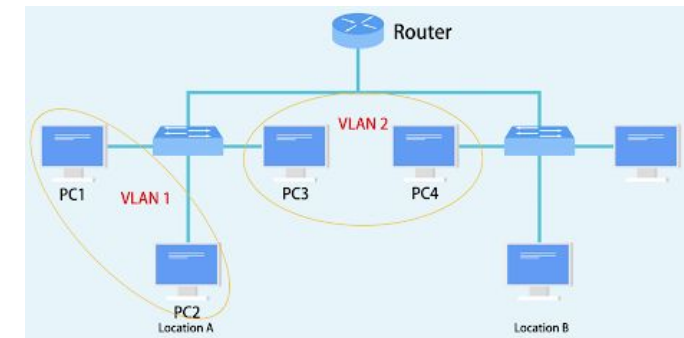


# Ethernet - Virtual LAN (VLAN)

- ❖ A broadcast domain that is partitioned and isolated in a LAN at the data link layer
- ❖ IEEE 802.1Q defines how virtual LANs operate on Ethernet networks using switches
  - It describes 4 extra bytes (VLAN tag) in the Ethernet frame
  - A VLAN has its own broadcast domain and data is only transmitted within the VLAN
    - To have communication between VLANs, routing is required. Like separate LANs
  - A node can be a member of multiple VLANs
  - VLAN frame length: 64 - 1522 bytes
  - VLAN is frequently used in the automotive industry

|                       |                 |                                   |                                 |                   |                              |                           |
|-----------------------|-----------------|-----------------------------------|---------------------------------|-------------------|------------------------------|---------------------------|
| Preamble<br>(7 bytes) | SFD<br>(1 byte) | Receiver MAC Address<br>(6 bytes) | Sender MAC Address<br>(6 bytes) | Type<br>(2 bytes) | Payload<br>(46 - 1500 bytes) | CRC Checksum<br>(4 bytes) |
|-----------------------|-----------------|-----------------------------------|---------------------------------|-------------------|------------------------------|---------------------------|

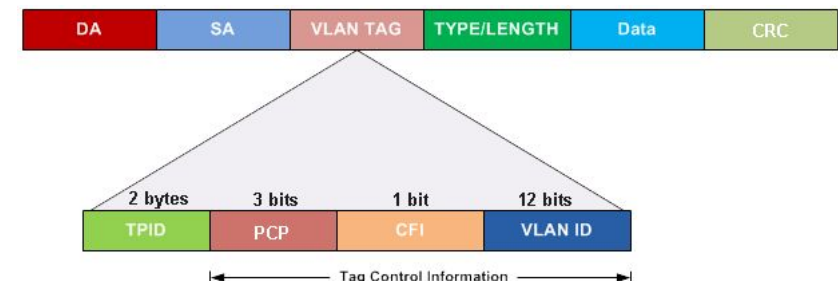
|                       |                 |                                   |                                 |                       |                   |                              |                           |
|-----------------------|-----------------|-----------------------------------|---------------------------------|-----------------------|-------------------|------------------------------|---------------------------|
| Preamble<br>(7 bytes) | SFD<br>(1 byte) | Receiver MAC Address<br>(6 bytes) | Sender MAC Address<br>(6 bytes) | VLAN Tag<br>(4 bytes) | Type<br>(2 bytes) | Payload<br>(42 - 1500 bytes) | CRC Checksum<br>(4 bytes) |
|-----------------------|-----------------|-----------------------------------|---------------------------------|-----------------------|-------------------|------------------------------|---------------------------|



E.g. In a school, VLAN1 for students and VLAN2 for teachers

# Ethernet - Virtual LAN (VLAN tag)

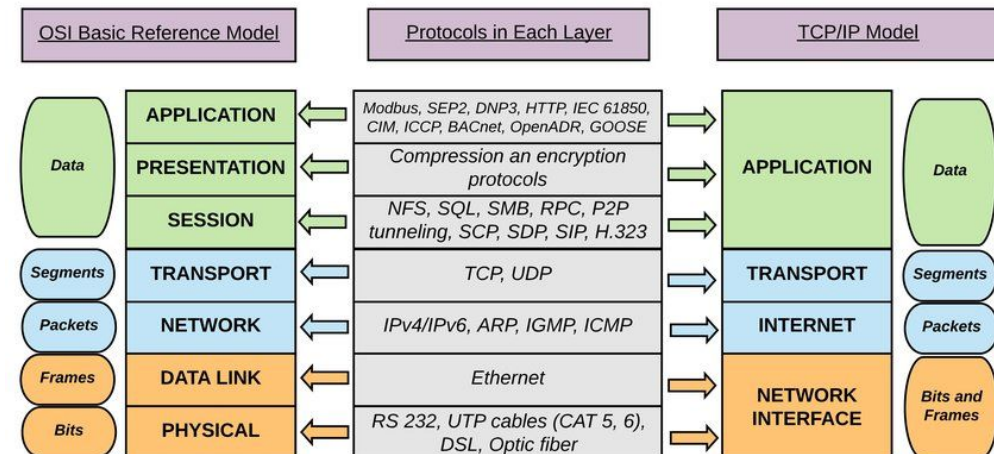
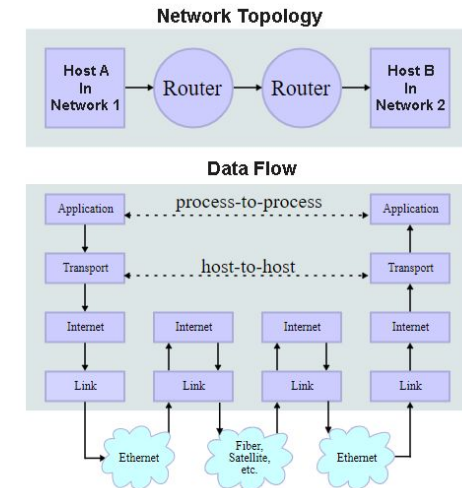
- ❖ VLAN tag is 4 bytes, divided into 2 fields
  - **Tag Protocol Identifier (TPID):** The ID of the IEEE 802.1Q protocol which is 0x8100 (2 bytes)
  - **Tag Control Information (TCI):** Contains information about the tag (2 bytes)
- ❖ Tag Control Information (TCI) consists of 3 fields
  - Priority Code Point (PCP)
    - It provides the option of defining priorities for messages to improve real-time behavior
  - Canonical Form Indicator (CFI): The IEEE802.1Q is only developed for Ethernet or Token Ring
    - This bit is 0 for Ethernet and 1 for Token Ring
  - VLAN ID: Identifier of the VLAN
    - 4094 possibilities. 0xFFF is reserved and 0x000 is used for NO VLAN (frames with priority)





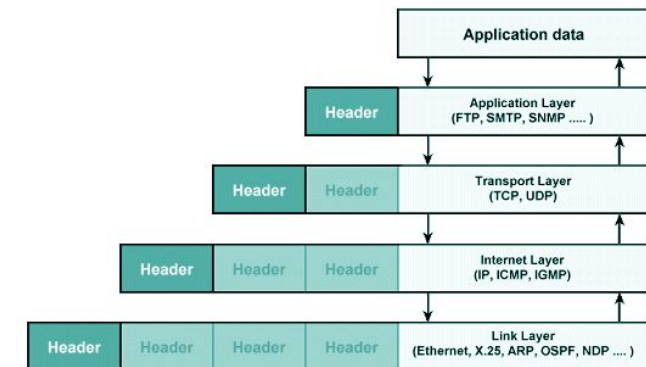
# Transmission Control Protocol / Internet Protocol (TCP/IP)

- ❖ A set of standard protocols designed for communication over internet (interconnected networks) consisting of different network segments that are linked by routers
- ❖ An internet (internetwork) is a very general concept
  - An internet is not limited in size. It can be
    - A couple of networks or
    - The worldwide Internet (capital I)
- ❖ A model which has 4 layers
  - But the central layers in this model are
    - The internet layer
    - The transport layer



# Internet Protocol - IPv4/IPv6

- ❖ Enables communication beyond the boundary of a local network (LAN)
  - By abstracting the lower layers (data link layer and physical layer) in a way that
    - A destination node can be reached in the same network or across multiple networks
- ❖ Defines the network layer of the OSI model
- ❖ IP is responsible to deliver a variable length data from one place to another by
  - Logical addressing of devices on the network uniquely - IP addresses
  - Creating IP packets by adding a header including source and destination IP addresses
  - Routing the IP packets over the internet
- ❖ Does not provide error detection & correction for the data field
  - But the header is protected by a checksum



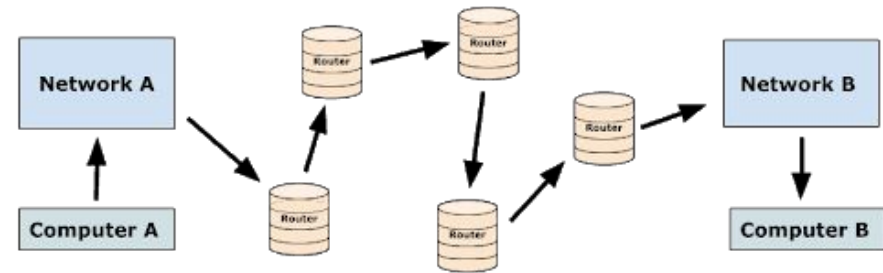
# Internet Protocol - IPv4/IPv6

## ❖ There are two versions of the Internet protocol

- IP version 4 - IPv4 (32-bit addresses)
- IP version 6 - IPv6 (128-bit addresses)

## ❖ Routers and IP Packet Routing

- In order to interconnect various networks, a router is used as a coupling element
- A router is a node that belongs to more than one network and has more than one IP addresses
- IP Routing is the process of determining the path that data follows in order to travel across multiple networks from its source to its destination



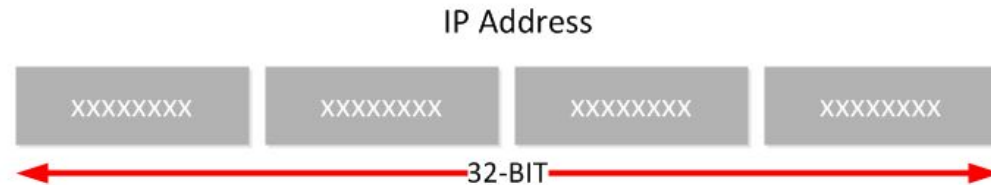
## ❖ IP is a connectionless protocol. No fixed physical connection is created

- Each packet may travel independently through a different route to the same destination.
- Fragmentation and reassembly of packets are done by the Internet Protocol

# Internet Protocol - IPv4

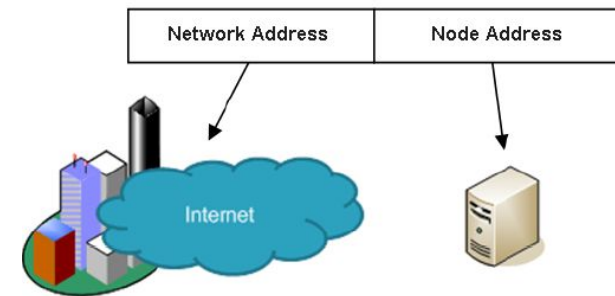
## ❖ 32-bit addresses and they can be written in different ways

- 208.80.154.224 (most used)
- 0xD0509AE0
- 0xD0.0x50.0x9A.0xE0
- 3494943456



## ❖ There are two types of address: General or Public and Local or private

- Public IP addresses can be accessed over the Internet
- Private IP addresses can't be accessed over the Internet and routers do not forward them to the Internet.
  - E.g. The IP addresses on the LAN of our school



## ❖ An IP address is composed of Network and Node Addresses

# Internet Protocol - IPv4

## ❖ Addresses and classes

- For public and private addresses there are different classes
- If an IP packet is to be sent to multiple nodes, both Multicast addresses and Broadcast addresses can be used
- The highest value of a host address range always corresponds to the associated broadcast address.  
E.g. 192.168.10.**255**

Public IPv4 Addresses

| Class | Start Address | End Address     | Properties                      |
|-------|---------------|-----------------|---------------------------------|
| A     | 0.0.0.0       | 127.255.255.255 | 127 networks / 16 777 214 nodes |
| B     | 128.0.0.0     | 191.255.255.255 | 16 384 networks / 65 534 nodes  |
| C     | 192.0.0.0     | 232.255.255.255 | 2 097 152 networks / 254 nodes  |
| D     | 224.0.0.0     | 239.255.255.255 | 268 435 456 multicast groups    |
| E     | 240.0.0.0     | 255.255.255.255 | Reserved                        |

Private IPv4 Addresses

| Class | Start Address | End Address     | Properties                   |
|-------|---------------|-----------------|------------------------------|
| A     | 10.0.0.0      | 10.255.255.255  | 1 network / 16 777 214 nodes |
| B     | 172.16.0.0    | 172.31.255.255  | 16 networks / 65 534 nodes   |
| C     | 192.168.0.0   | 192.168.255.255 | 256 networks / 254 nodes     |

# Internet Protocol - IPv4

---

## ❖ Special IP addresses

- 127.0.0.1 - localhost (loopback)
- 169.254.x.x - auto generated when no address is given
- IP used by DNS servers. E.g. 8.8.8.8 - Google DNS

## ❖ Subnetting and Subnet Masks

- Subnetting is a technique to make several subnets from a given IP address
- The router filters the subnet addresses from the IP address by means of subnet masks
  - The bits that represent the subnet address are 1s
  - The bits that represent the host addresses are 0s
- E.g. a company works with IP address 172.23.0.0 (class B)
  - Subnet mask is 255.255.0.0 or 1111 1111 1111 1111 0000 0000 0000 0000



# Internet Protocol - IPv4

## ❖ If the entire company has to be divided into 10 different subnets.

- All subnets can be linked to each other via a router
  - With 4 bits, 16 different combinations can be created.
  - The required 10 subnets can be created by adding 4 bits to the Network address
  - Subnet mask is 1111 1111 1111 1111 **1111** 0000 0000 0000

Table 3.4: Subnetting and subnet masks

| BYTE 3<br>(binary code) | BYTE 3<br>(decimal value) | Subnet       | Subnetmask    |
|-------------------------|---------------------------|--------------|---------------|
| 0000 0000               | 0                         | 172.23.0.0   | 255.255.240.0 |
| 0001 0000               | 16                        | 172.23.16.0  | 255.255.240.0 |
| 0010 0000               | 32                        | 172.23.32.0  | 255.255.240.0 |
| 0011 0000               | 48                        | 172.23.48.0  | 255.255.240.0 |
| 0100 0000               | 64                        | 172.23.64.0  | 255.255.240.0 |
| 0101 0000               | 80                        | 172.23.80.0  | 255.255.240.0 |
| 0110 0000               | 96                        | 172.23.96.0  | 255.255.240.0 |
| 0111 0000               | 112                       | 172.23.112.0 | 255.255.240.0 |
| 1000 0000               | 128                       | 172.23.128.0 | 255.255.240.0 |
| 1001 0000               | 144                       | 172.23.144.0 | 255.255.240.0 |

## ❖ Classless Inter-Domain Routing (CIDR)

- A new way of addressing for the Internet which can lead to a more efficient use of IP addresses in comparison to the classes A,B and C
- The Network address is not limited to 8,16 or 24 bits
- A CIDR address contains the 32-bit IP address and the number of bits that are part of the network address. E.g. 192.168.0.0/28

Network: 192.168.0.0/28  
Broadcast: 192.168.0.15  
HostMin: 192.168.0.1  
HostMax: 192.168.0.14  
Hosts/Net: 14

# Internet Protocol - IPv6

---

- ❖ IPv6 (128-bit addresses) was mainly developed because of
  - The shortage of addresses of IPv4
  - Optimization of the routing process
    - Comparing to IPv4, the number of fields in the IPv6 header has been reduced from 12 to 8 fields
- ❖ IPv6 addresses are written by grouping two bytes in hex format separated by a colon
- ❖ Four zeros are typically written as only one zero or are completely omitted
- ❖ Examples of IPv6 addresses:
  - 1080:0:0:0:8:800:200C:4170 or 1080::8:800:200C:4170
  - FF01:0:0:0:0:0:0:101 or FF01::101
  - 0:0:0:0:0:0:0:1 or ::1(loopback address)
- ❖ IPv4 addresses can also be implemented in IPv6. E.g. ::FFFF:129.140.55.138

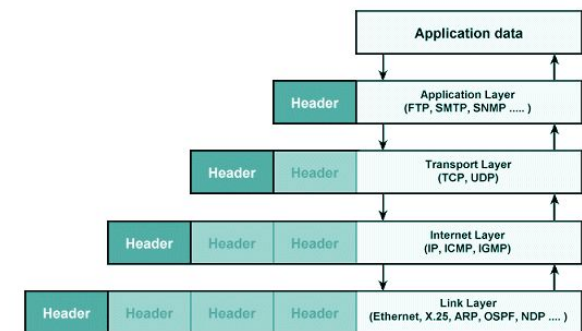
# Internet Protocol - Extension protocols

---

- ❖ There are some extension protocols in the internet layer which are supporting IP
  - The Dynamic Host Configuration Protocol (DHCP)
    - It is able to automatically assign IP addresses to nodes
  - The Internet Control Message Protocol (ICMP)
    - Is part of every IP implementation and is used for control tasks
    - Typical application examples are **ping** and **traceroute**
    - Size of control messages is 48 bits
  - The Address Resolution Protocol (ARP)
    - It is used to determine the relation between IP and MAC addresses.
    - If an IP node does not know the MAC address of a destination, it can be requested using ARP
    - The node broadcasts an ARP request to the network, the response contains the MAC address
  - And etc.

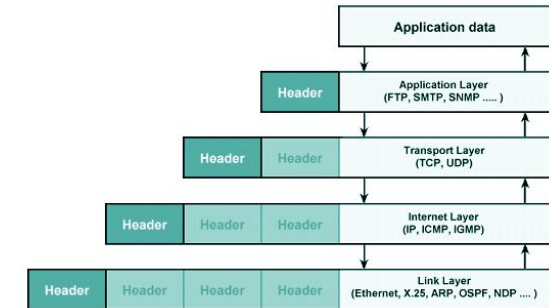
# Transport layer - TCP and UDP

- ❖ There are two transport protocols in Layer 4 of the OSI model:
  - Transmission Control Protocol (TCP) - connection-oriented transmission
  - User Datagram Protocol (UDP) - connectionless transmission
- ❖ Both protocols split the data that are to be transmitted into smaller parts.
  - These splitted data are called segments in TCP and datagrams in UDP
- ❖ To reach the destination node, the Transport layer PDUs are delivered to the Internet Protocol layer to transmit.
- ❖ To address the services and applications in the upper layers,
  - **64k Ports** are provided
  - An open port allows a bound service to listen for, accept connections, and exchange data over the network



# Transport layer - User Datagram Protocol (UDP)

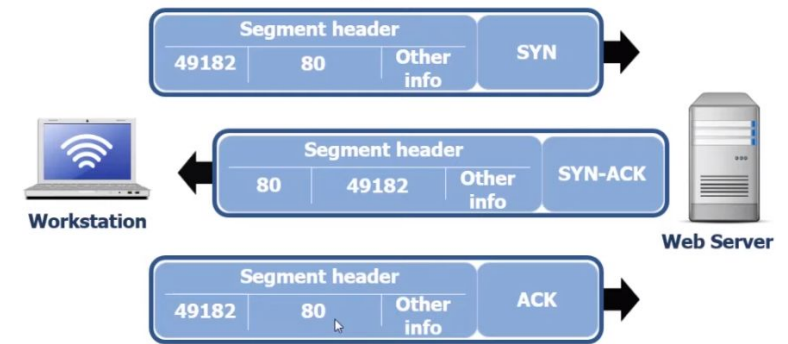
- ❖ Is a connectionless transport protocol that transports datagrams
- ❖ There is no flow & error control to ensure proper data transmission
  - There is no mechanism to guarantee data transmission
  - There is no error detection and correction mechanism
  - There is no packet order checking
  - If it is required, such mechanisms can be implemented in the higher layer
  - E.g. Voice Over IP (VOIP)
- ❖ Advantage: Faster than TCP and can be used to send datagrams as Multicast or Broadcast
- ❖ Application data is encapsulated into datagrams by adding a header with source and destination ports, then passed to the Internet Protocol for routing and optional fragmentation
- ❖ Decapsulation of datagrams is done and the resulting data is delivered to the application layer.



# Transport layer - Transmission Control Protocol (TCP)

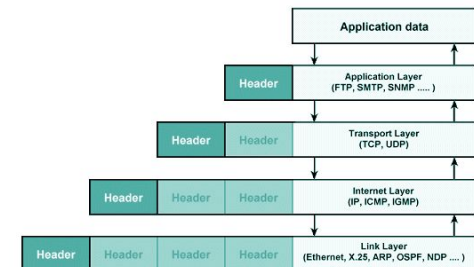
- ❖ TCP represents a connection-oriented transport protocol
  - A connection between two nodes must be established before the actual data transmission
  - The nodes are identified by IP addresses and port numbers
- ❖ Stateful connection: A three-way handshake: SYN, SYN-ACK, ACK
- ❖ Data arrives in order and duplicate data is discarded
- ❖ Reliable data transmission.
- ❖ Lost or corrupted segments are resent
- ❖ Segments are protected by CRC checksum
- ❖ Broadcasting and multicasting are not possible.
  - Because a connection between two nodes is required.

TCP - 3 way Handshaking to establish the connection



# Transport layer - Transmission Control Protocol (TCP)

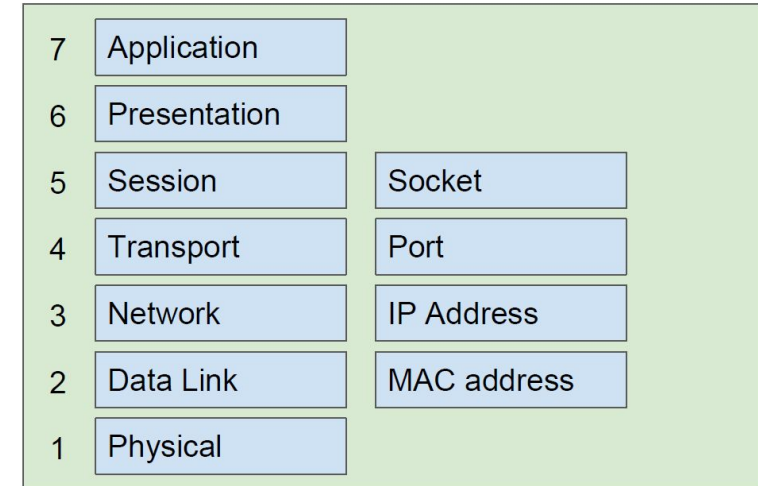
- ❖ Includes traffic congestion control
- ❖ Acknowledgment is used for every segment
  - If no acknowledgment within a defined time period, the segment will be sent again
- ❖ When all data being exchanged and acknowledged, the connection can be closed
  - A node sends a segment with set **FIN** flag, which signals the connection termination
  - The receiver of this **FIN** segment must respond with an acknowledgment
  - If the receiver also wants to terminate the connection, it sends a **FIN** segment to the original sender. If this is also acknowledged, the connection termination is complete
- ❖ Application data is encapsulated into segments by adding a header including sequence numbers, and source and destination ports, then passed to the Internet Protocol for routing and optional fragmentation
- ❖ Decapsulated segments are delivered as payload to the application layer





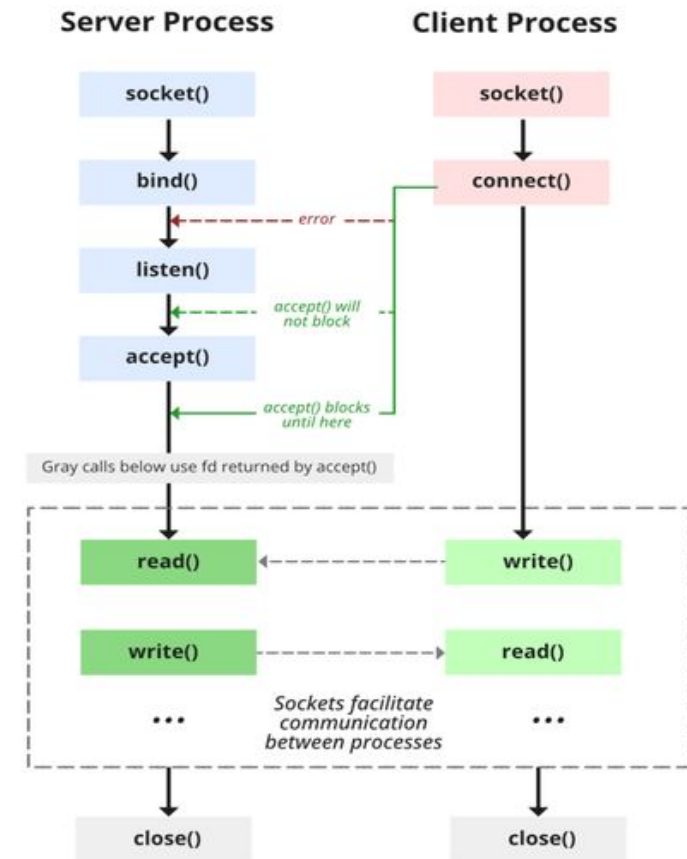
# The OSI Model Addressing - TCP/IP

- ❖ Ports are service point addresses which are used by applications and services.
- ❖ UDP/TCP/IP provides 64k port numbers
- ❖ Ports 0-1023 as standardized ports, known as system ports
- ❖ Example of such ports:
  - 20-21: FTP - File Transfer Protocol
  - 22: SSH - Secure Shell
  - 53: DNS - Domain Name Service
  - 80: HTTP - HyperText Transfer Protocol
  - 123: NTP - Network Time Protocol
  - 443: HTTPS - HTTP Secure
- ❖ A socket is an endpoint (combination of an IP address and a port number) of a two-way communication. Sockets are provided by the session layer; A socket looks like a file descriptor.



# Socket Programming - Socket

- ❖ An interface between application and network
  - An application creates a socket
  - The socket type dictates the style of communication
    - connection-oriented or connectionless
- ❖ Once configured the application can
  - Pass data to the socket for network transmission
  - Receive data from the socket (transmitted by some other host)
- ❖ Sockets work with I/O services just like files.
- ❖ Sockets have special needs:
  - Establishing a connection
  - Specifying communication endpoint addresses



[State diagram for server and client model of Socket](#)

# Socket Programming - Socket

## ❖ Socket Descriptor Data Structure:

- Family: AF\_INET
- Service: SOCK\_STREAM
- Local IP: 111.22.3.4
- Remote IP: 123.45.6.78
- Local Port: 2249
- Remote Port: 3726

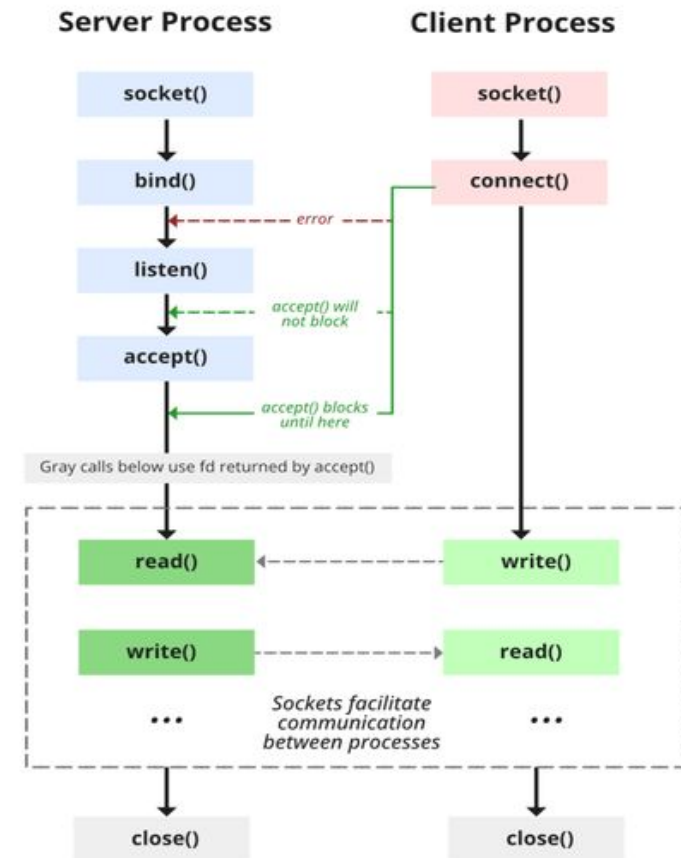
## ❖ Types of socket:

### SOCK\_STREAM (TCP)

- Reliable delivery
- In-order is guaranteed
- Connection-oriented
- Bidirectional

### SOCK\_DGRAM (UDP)

- Unreliable delivery
- No order is guaranteed
- Connectionless
- Can send/receive



# Socket Programming - Create Socket

---

- ❖ `int s = socket(domain, type, protocol);` // it just creates the interface
  - `s`: socket descriptor, an integer (like a file descriptor)
  - `domain`: integer, communication domain
    - E.g., `PF_INET` (IPv4 protocol) – typically used
    - In Linux: `#define PF_INET AF_INET` (value of 2)
  - `type`: communication type
    - `SOCK_STREAM`: reliable and connection-oriented
    - `SOCK_DGRAM`: unreliable, connectionless
    - other values: need root permission, rarely used, or obsolete
  - `protocol`: specifies protocol (see `/etc/protocols` for a list of options)
    - Usually set to 0 (`IPPROTO_IP`)

# Socket Programming - bind

---

- ❖ The **socket()** system call returns a socket descriptor (small integer) or -1 on error.
- ❖ **socket()** allocates resources needed for a communication endpoint.
- ❖ A socket provides an interface to send data to/from the network through a port
- ❖ The **bind** function associates and reserves a port for use by the socket
- ❖ `int status = bind(sockid, &addrport, size);`
  - status: error status, 0 if successful or -1 on error.
  - sockid: integer, socket descriptor
  - addrport: struct sockaddr, the (IP) address and port of the machine
    - address usually set to `INADDR_ANY` – chooses a local address
  - size: the size (in bytes) of the addrport structure

# Socket Programming - bind

---

- ❖ The **bind()** system call is used to assign an address to an existing socket.
- ❖ Assigns the address specified by the **sockaddr** structure to the socket descriptor.
- ❖ You can bind() an instance of the **sockaddr\_in** structure:
  - `bind( mysock, (struct sockaddr*) &myaddr, sizeof(myaddr) );`
- ❖ For example:
  - `int mysock,err; struct sockaddr_in myaddr;`
  - `mysock = socket(PF_INET,SOCK_STREAM,0);`
  - `myaddr.sin_family = AF_INET;`
  - `myaddr.sin_port = htons( portnum );`
  - `myaddr.sin_addr = htonl( ipaddress);`
  - `err = bind(mysock, (sockaddr *) &myaddr, sizeof(myaddr));`

# Socket Programming - bind

---

## ❖ There are a number of uses for bind():

- Server would like to bind to a well known address (port number).
- Client can bind to a specific port or ask the O.S. to assign any available port number.
- Clients typically don't care what port they are assigned.
- When you call bind, you can tell it to assign you any available port: `myaddr.port = htons(0);`
- To specify your IP address you can use `INADDR_ANY`,

## ❖ Possible to skip bind

- `SOCK_DGRAM`:
  - If only sending, no need to bind. The OS finds a port each time the socket sends
  - If receiving, need to bind
- `SOCK_STREAM`:
  - Destination determined during connection setup. Don't need to know port sending from



# Socket Programming - Connection Setup (SOCK\_STREAM)

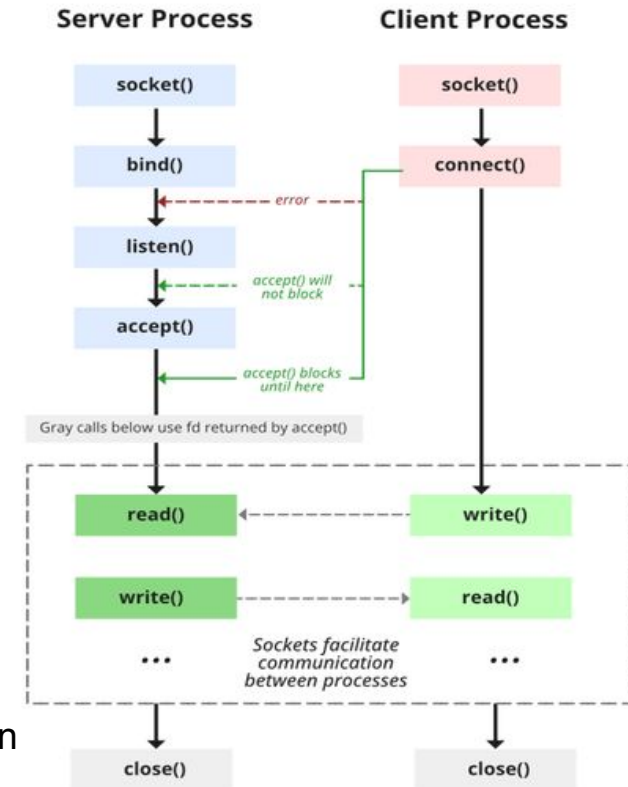
- ❖ No connection setup for SOCK\_DGRAM
- ❖ A connection occurs between two kinds of participants
  - Server: waits for a client to request a connection
  - Client: initiates a connection request to the server
- ❖ Once a connection is established, both the participants can send & receive data, and terminate the connection
- ❖ Connection setup steps

## Server:

- Step 1: listen (for incoming requests)
- Step 3: accept (a request)
- Step 4: data transfer

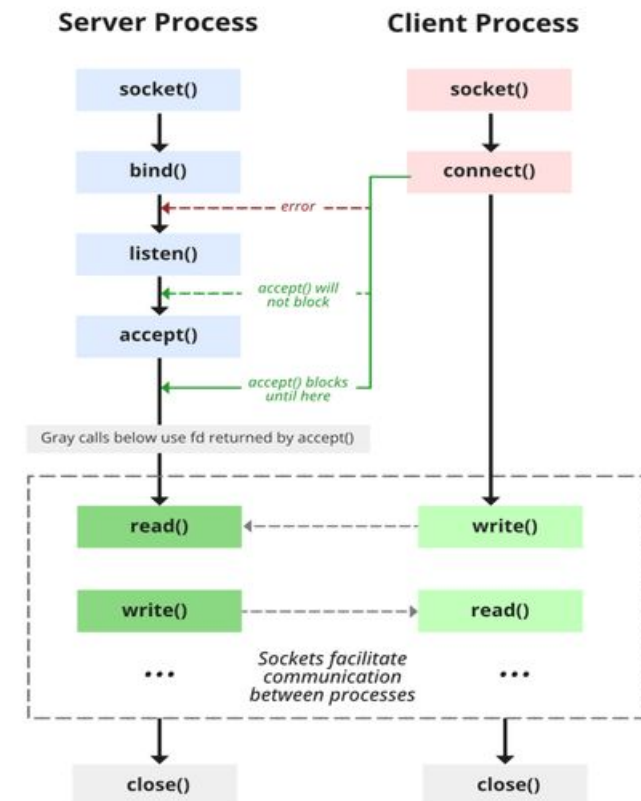
## Client:

- Step 2: request & establish connection
- Step 4: data transfer



# Socket Programming - Connection Setup (SOCK\_STREAM)

- ❖ The accepted connection is on a new socket. The old socket continues to listen
- ❖ Connection setup: listen & accept (called by the server)
  - `int status = listen(sock, queuelen);`
    - status: 0 if listening, -1 if error
    - sock: integer, socket descriptor
    - queuelen: integer, # of clients that can “wait” for a connection
    - listen is non-blocking: returns immediately
  - `int s = accept(sock, &name, &namelen);`
    - s: integer, the new socket (used for data-transfer)
    - sock: integer, the orig. socket (being listened on)
    - name: **struct sockaddr**, address of the client participant
    - namelen: sizeof(name): value/result parameter
      - must be set appropriately before call
    - accept is blocking: waits for connection before returning



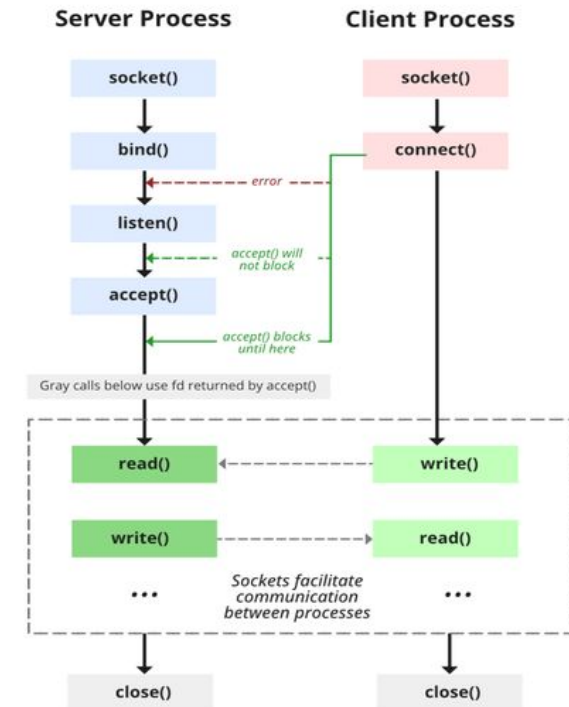
# Socket Programming - Connection Setup (SOCK\_STREAM)

## ❖ Connect call by the client

- `int status = connect(sock, &name, namelen);`
  - status: 0 if successful connect, -1 otherwise
  - sock: integer, socket to be used in connection
  - name: struct sockaddr: address of the server
  - namelen: integer, sizeof(name)
  - connect is blocking

## ❖ With a connection (SOCK\_STREAM):

- `int count = write(sock, buf, len);`
  - count: # bytes transmitted
    - 0: The connection was closed by the remote host.
    - -1: The write system call was interrupted, or failed for some reason.
    - n: The write system call wrote 'n' bytes into the socket..
  - buf: void \*, buffer to be transmitted
  - len: integer, length of buffer (in bytes) to transmit



# Socket Programming - Sending / Receiving Data

---

## ❖ With a connection (SOCK\_STREAM):

- `int count = read(sock, buf, len);`
  - `count`: # bytes received (-1 if error)
    - 0: The connection was closed by the remote host.
    - -1: The read system call was interrupted, or failed for some reason.
    - `n`: The read system call put '`n`' bytes into the buffer we supplied it with.
  - `buf`: `void *`, stores received bytes
  - `len`: integer, length of buffer (in bytes) to receive
- `int count = send(sock, buf, len, flags);`
  - `count`: # bytes transmitted (-1 if error)
  - `buf`: `void *`, buffer to be transmitted
  - `len`: integer, length of buffer (in bytes) to transmit
  - `flags`: integer, special options, usually just 0

# Socket Programming - Sending / Receiving Data

---

## ❖ With a connection (SOCK\_STREAM)

- `int count = recv(sock, buf, len, flags);`
  - `count`: # bytes received (-1 if error)
  - `buf`: `void *`, stores received bytes
  - `len`: integer, length of buffer (in bytes) to receive
  - `flags`: integer, special options, usually just 0

## ❖ To Send Data (SOCK\_DGRAM)

- `int count = sendto(sock, buf, len, flags, addr, addrlen);`
  - `count`, `sock`, `buf`, `len`, `flags`: same as `send`
  - `addr`: `struct sockaddr *`, address of the destination
  - `addrlen`: `sizeof(struct sockaddr)`

# Socket Programming - Close

---

## ❖ To Receive Data (SOCK\_DGRAM)

- `int count = recvfrom(sock, buf, len, flags, addr, &addrlen);`
  - `count`, `sock`, `buf`, `len`, `flags`: same as `recv`
  - `addr`: `struct sockaddr *`, address of the source
  - `addrlen`: `sizeof(struct sockaddr)`: value/result parameter

## ❖ **recvfrom**, **sendto**, **send**, **recv**, **read** and **write** calls are blocking

- Returns only after data is sent (to socket buffer) / received

## ❖ Closing a socket

- When finished using a socket, the socket should be closed
- Closes a connection (for `SOCK_STREAM`) and frees up the port used by the socket
- `status = close(s);`
  - `status`: 0 if successful, -1 if error
  - `s`: the file descriptor (socket being closed)

# Socket Programming - The struct sockaddr

---

- ❖ The generic: `struct sockaddr { u_short sa_family; char sa_data[14]; };`
  - `sa_family`
    - Specifies which address family is being used
    - Determines how the remaining 14 bytes are used
- ❖ The Internet-specific:
  - `struct sockaddr_in { short sin_family; u_short sin_port; struct in_addr sin_addr; char sin_zero[8]; };`
    - `sin_family = AF_INET`
    - `sin_port`: port # (0-65535)
    - `sin_addr`: IP-address
    - `sin_zero`: unused
- ❖ We don't need to deal with `sockaddr` structures since we will only deal with a real protocol family.
- ❖ We can use the `sockaddr_in` structure.

# Socket Programming - Network Byte Order

---

- ❖ All values stored in a `sockaddr_in` must be in network byte order.
  - `u_short sin_port` (16 bits): a TCP/IP port number.
  - `in_addr sin_addr` (32 bits): an IP address.
- ❖ Different machines may communicate with each others over a network
- ❖ Different machines / OS's use different byte orderings
  - little-endian: lower bytes first
  - big-endian: higher bytes first
- ❖ Byte-ordering functions (if little-endian convert to big-endian)
  - `uint32_t htonl(uint32_t hostlong);`
  - `uint16_t htons(uint16_t hostshort);`
  - `uint32_t ntohl(uint32_t netlong);`
  - `uint16_t ntohs(uint16_t netshort);`
  - Look at: <https://linux.die.net/man/3/htonl>



# Ethernet and TCP/IP

---

## ❖ Some useful links

- [How Ethernet Works and IEEE 802.3 Specification](#)
- [How the Internet Protocol \(IP\) Works](#)
- [How the Transmission Control Protocol \(TCP\) Works](#)
- [Ethernet Basics](#)
- [Subnetting explained](#)
- [Network IDs and Subnet Masks](#)
- Socket Programming Tutorials In C For Beginners ([Part 1](#) and [Part 2](#))
- [Running the Winsock Client and Server Code Sample](#)
- [Multicast IPv4 address to MAC address mapping](#)
- [Socket programming in C on Linux](#)